



THE UNIVERSITY OF  
**SYDNEY**

# **OVER-SAVE: Project Report**

ELEC5619: Object Oriented Application Frameworks

Tuesday 5pm Group 3

|                |             |
|----------------|-------------|
| Kartik Nair    | (510611749) |
| Antriksh Dhand | (510415022) |
| Taozhao Chen   | (510019015) |
| Joey Chen      | (520076428) |
| Aryan Adhikari | (510615596) |

## Contents

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                       | <b>3</b>  |
| 1.1      | Overview . . . . .                                        | 3         |
| 1.2      | Aims . . . . .                                            | 3         |
| 1.3      | Project scope . . . . .                                   | 4         |
| 1.4      | Project scenarios . . . . .                               | 4         |
| 1.5      | Audiences and primary users . . . . .                     | 4         |
| <b>2</b> | <b>Architecture</b>                                       | <b>6</b>  |
| 2.1      | UML class diagram . . . . .                               | 6         |
| 2.1.1    | Core domain models and key relationships . . . . .        | 6         |
| 2.2      | Use case diagram . . . . .                                | 8         |
| 2.2.1    | Actor analysis . . . . .                                  | 10        |
| <b>3</b> | <b>Functionalities</b>                                    | <b>11</b> |
| 3.1      | Project functionalities . . . . .                         | 11        |
| 3.1.1    | FR-01: User registration . . . . .                        | 12        |
| 3.1.2    | FR-02: Email verification and OAuth integration . . . . . | 13        |
| 3.1.3    | FR-03: Login . . . . .                                    | 14        |
| 3.1.4    | FR-04: Logout . . . . .                                   | 15        |
| 3.1.5    | FR-05: Password reset . . . . .                           | 16        |
| 3.1.6    | FR-06: Session management . . . . .                       | 17        |
| 3.1.7    | FR-07: Expense recording . . . . .                        | 18        |
| 3.1.8    | FR-08: Income recording . . . . .                         | 19        |
| 3.1.9    | FR-09: Category management . . . . .                      | 20        |
| 3.1.10   | FR-10: Recurring subscriptions . . . . .                  | 21        |
| 3.1.11   | FR-11: Budget setting . . . . .                           | 22        |
| 3.1.12   | FR-12: Savings goals . . . . .                            | 23        |
| 3.1.13   | FR-13: Goal contributions . . . . .                       | 24        |
| 3.1.14   | FR-14: Dashboard display . . . . .                        | 25        |
| 3.1.15   | FR-15: Savings progress view . . . . .                    | 26        |
| 3.1.16   | FR-16: Data export . . . . .                              | 27        |
| 3.1.17   | FR-17: Data import . . . . .                              | 28        |
| 3.1.18   | FR-18: Email notifications . . . . .                      | 29        |
| 3.1.19   | FR-19: BudgetCoin reward system . . . . .                 | 30        |
| 3.2      | Non-functional requirements achieved . . . . .            | 31        |
| 3.2.1    | Performance . . . . .                                     | 31        |
| 3.2.2    | Security . . . . .                                        | 32        |
| 3.2.3    | Usability . . . . .                                       | 34        |

---

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 3.2.4    | Reliability . . . . .                           | 36        |
| 3.2.5    | Maintainability . . . . .                       | 39        |
| <b>4</b> | <b>Spring and System Integration</b>            | <b>42</b> |
| 4.1      | External APIs . . . . .                         | 42        |
| 4.2      | Spring Controllers and REST APIs . . . . .      | 42        |
| 4.3      | System integration and data flow . . . . .      | 46        |
| <b>5</b> | <b>Testing Strategy and Implementation</b>      | <b>48</b> |
| 5.1      | Testing strategy and scope . . . . .            | 48        |
| 5.2      | Testing structure and implementation . . . . .  | 48        |
| 5.2.1    | Unit testing . . . . .                          | 48        |
| 5.2.2    | Integration and use case testing . . . . .      | 48        |
| 5.2.3    | Acceptance testing . . . . .                    | 49        |
| 5.3      | Coverage monitoring and Jacoco report . . . . . | 49        |
|          | <b>References</b>                               | <b>50</b> |

# 1 Introduction

## 1.1 Overview

In today's economic climate, financial stress is an increasingly common challenge, especially among young adults, university students, and low-income earners. Rising living costs, unpredictable expenses, and limited financial literacy make it difficult for many to manage their personal budgets effectively. For example, a 2024 survey found that 55% of Australians aged 15-19 are now very concerned about their financial security (Mission Australia, 2023). Another national report showed that 37% of 23-year-olds reported financial stress in 2023, up from 30% the year prior (National Centre for Vocational Education Research (NCVER), 2023). As international students ourselves, we have witnessed firsthand how easily small oversights – such as forgetting a subscription payment or misjudging monthly spending – can lead to significant stress and financial instability.

*OVER-SAVE* is a web-based budget management application designed to empower individuals to take control of their finances through a simple, engaging, platform with built-in gamification and positive reinforcement. The system allows users to record income and expenses, track recurring subscriptions (such as Netflix or Spotify), define monthly budgets, and set savings goals. In addition, users can import bank transactions, export financial summaries, and receive monthly report emails that visualise their spending habits. A unique gamification element in the form of in-app currency *BudgetCoins* encourages consistent engagement by rewarding users for responsible financial behaviour, such as adding transactions or reaching savings goals. These coins can later be redeemed for real-world rewards, providing tangible incentives for those taking care of their finances.

## 1.2 Aims

The primary aim of *OVER-SAVE* is to create an accessible and engaging platform that encourages better financial habits among users who may otherwise lack the tools or motivation to budget consistently. Specifically, the project seeks to:

- Reduce barriers to financial management for lower-income earners and students through a clean, intuitive interface.
- Encourage long-term engagement through gamified incentives and progress tracking.
- Improve user awareness of spending patterns via interactive dashboards and monthly summary reports.
- Promote responsible financial behaviour through real-time feedback and positive reinforcement.

Overall, the application aims to make financial management not just easier, but more rewarding and sustainable.

### 1.3 Project scope

The scope of the OVER-SAVE project covers both core and advanced budgeting functionalities. The system supports:

- User authentication via secure login and registration.
- Recording, editing, and categorising income and expense transactions.
- Setting and monitoring budgets and savings goals.
- Tracking recurring subscriptions with reminders.
- Importing and exporting transaction data in CSV format.
- Generating automated monthly summary emails with visual insights.
- Gamified rewards through BudgetCoins, which can be redeemed for prizes.

The project excludes advanced financial forecasting, AI-based recommendations, and integrations with third-party banking APIs. The focus is on simplicity, reliability, usability, and motivation through clear design and consistent user feedback.

### 1.4 Project scenarios

Typical usage scenarios include:

1. Daily expense tracking: A university student creates a new category called “Eating out” and records small purchases throughout the week to stay within their food budget.
2. Subscription awareness: A part-time worker adds their monthly streaming subscriptions to visualise recurring costs and plan cancellations if necessary.
3. Savings motivation: A user sets a savings goal for an upcoming trip and tracks progress, earning BudgetCoins as they contribute regularly.
4. Monthly reflection: At the end of every month, the system sends a monthly summary to the user highlighting key spending categories and goal progress.

These scenarios demonstrate how the system supports both short-term awareness and long-term financial stability through consistent engagement and feedback.

### 1.5 Audiences and primary users

OVER-SAVE is primarily designed for:

- University students, particularly international students managing limited budgets and fluctuating income.

- Young professionals or low-income earners seeking a simple way to monitor expenses and build savings habits.
- Financial literacy programs or community initiatives that require accessible tools for teaching personal finance.

Secondary stakeholders include educators promoting financial education, community organisations addressing economic wellbeing, and potential sponsors providing real-life rewards for BudgetCoin redemptions. By focusing on accessibility, simplicity, and positive reinforcement, OVER-SAVE aims to foster healthier financial habits among those most vulnerable to financial stress.

## 2 Architecture

This project is a budget tracking and personal finance management system, designed using a typical layered architecture pattern. The system architecture is illustrated through class diagrams and use case diagrams.

### 2.1 UML class diagram

The UML class diagram (Figure 1) presents the static structure of the system, including entities, components, and their relationships. The system adopts a three-tier architecture to ensure separation of concerns, high cohesion, and low coupling. The core components of each layer are as follows:

- **Controller:** Handles HTTP requests, performs input validation, and delegates to the service layer. Key components include `Controller`, `ExpenseController`, and `AccountController`.
- **Service:** Implements core business logic, transaction management, and data aggregation. Key services include `AuthService`, `BudgetService`, `ExpenseService`, and `ReportService`.
- **Data:** Manages database interactions and data persistence. Key repositories include `UserRepository`, `CashFlowRepository`, and `AccountRepository`.

#### 2.1.1 Core domain models and key relationships

The core domain model revolves around users, cash flow, and budget settings.<sup>1</sup>

##### User and authentication

- `User` is the central entity, associated with all core data such as `Account`, `Category`, and `Goal`.
- `Session` tracks user session states and is managed by `SessionManagementService` to ensure system security.
- `PasswordResetToken` and `LoginAttempt` enhance authentication and security mechanisms.

##### Accounts and cash flow

- `Account` represents financial sources (e.g. bank cards, cash) and maintains real-time balance via the `balance` attribute.

---

<sup>1</sup>Due to the complexity of the class diagram, this section provides a brief written overview. The complete class diagram is presented in Figure 1, with additional detailed snapshots included in the Appendix for reference.

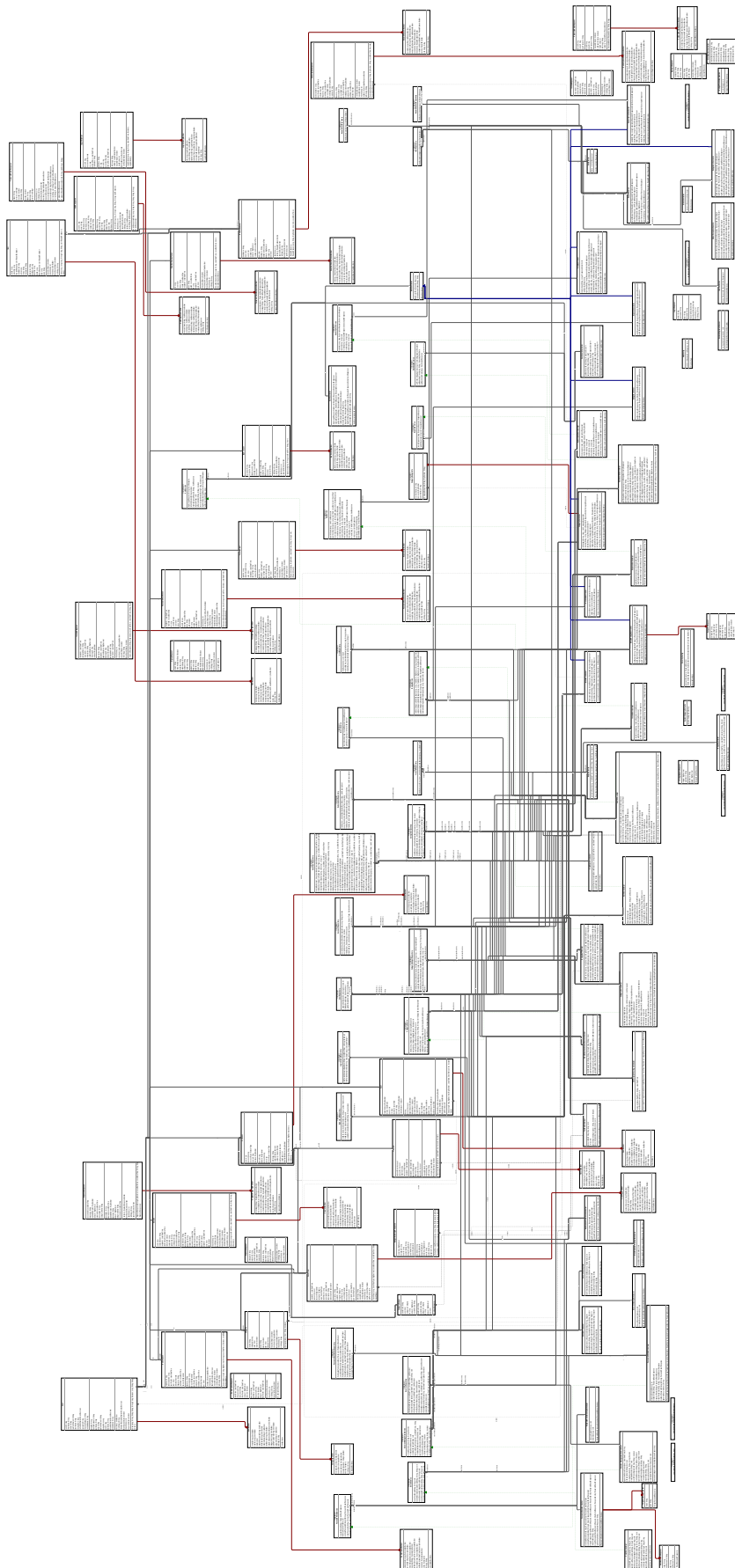


Figure 1: UML class diagram of OVER-SAVE



- **CashFlow** abstracts all transactions (income and expenses), linked to specific **Account** and **Category**, serving as the basis for budget comparison.
- **Transfer** records fund transfers between accounts.

### Budget and goals

- **CategoryBudget** stores user-defined budgets for specific categories and months, forming the core of budget control.
- **Subscription** captures recurring transactions, useful for automated forecasting and budget planning.
- **Goal** tracks long-term financial objectives, linked to **Account** to monitor fund accumulation.

### Service and repository

- Each core business entity is supported by corresponding services and repositories. For example, **BudgetService** depends on **CategoryBudgetRepository** and **CashFlowRepository**.
- The service layer extensively uses dependency injection; i.e. defining interfaces (e.g. **BudgetService**, **AuthService**) and separating them from their implementation classes (e.g. **BudgetServiceImpl**, **AuthServiceImpl**). This significantly improves testability and scalability.

## 2.2 Use case diagram

The use case diagram (Figure 2) illustrates the functional scope and actor interactions for the OVER-SAVE personal finance management application. There are four main categories of use cases:

### 1. Authentication and account management

- Login
- Register account
- Reset password
- Register with Google

### 2. Core financial operations

- Add expense
- Add income

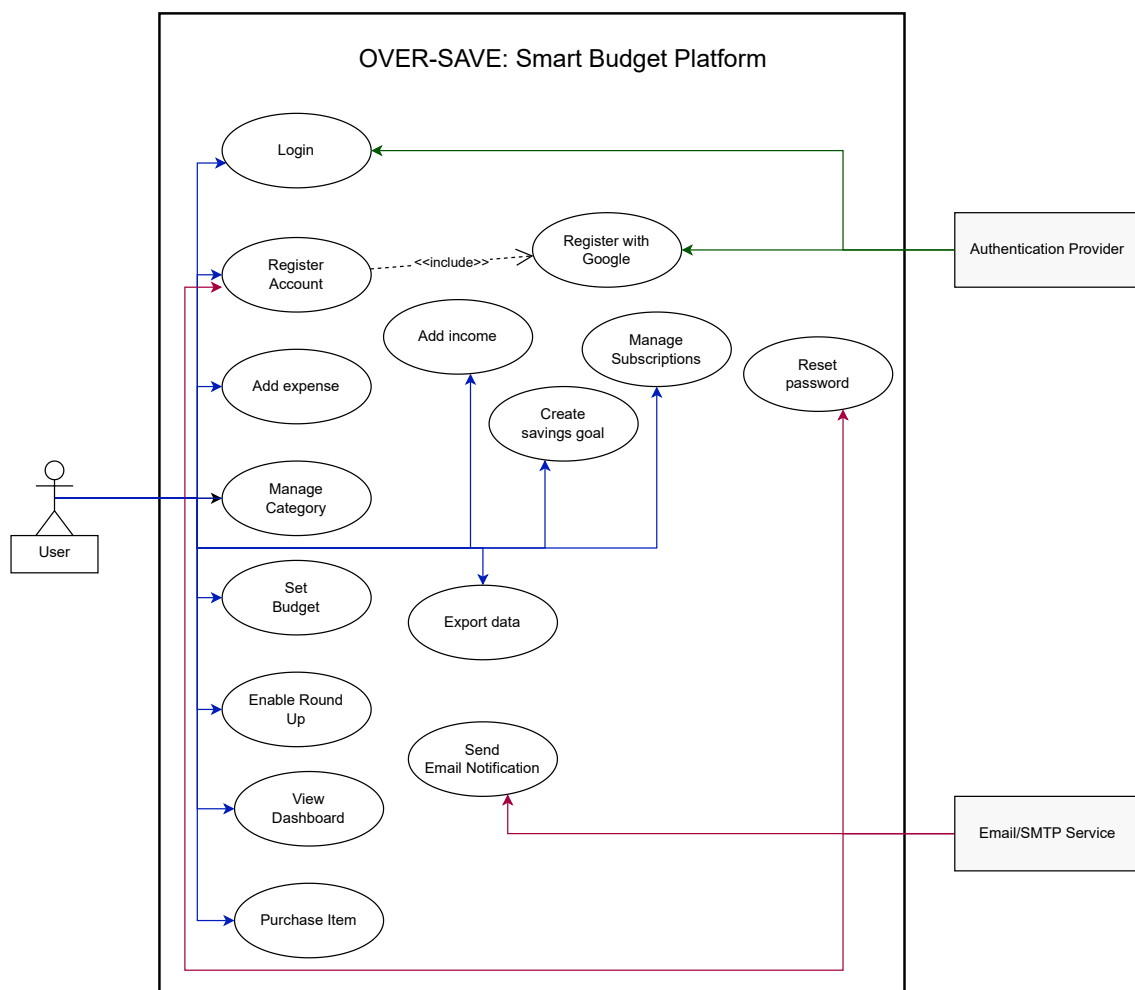


Figure 2: Use case diagram for the OVER-SAVE application

- Manage categories
  - Manage subscriptions
  - Create savings goal
  - Import transactions
3. Budget and planning
- Set budget
4. Analytics and reporting
- View dashboard
  - Export data
  - Send email notification

### 2.2.1 Actor analysis

#### Primary Actor

- User: The main actor representing individuals managing their personal finances. Connected via blue lines to the majority of use cases, demonstrating the user-centric nature of the application.

#### Secondary Actors

- Authentication Provider (top-right): External identity services (Google, Apple) that handle secure user authentication. Connected via green lines to authentication-related use cases including Login, Register Account, and Reset Password.
- Email/SMTP Service (bottom-right): External email services that enable system-to-user communication. Connected via orange lines to notification and verification use cases.

## 3 Functionalities

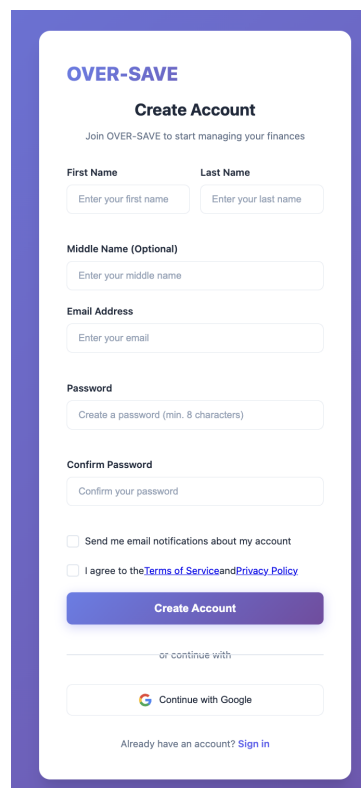
### 3.1 Project functionalities

This section will aim to clearly state what functionalities we have implemented in the OVER-SAVE application. Alongside each description, a short justification is provided as to how this functionality supports the goal of our project.

#### Contents

|        |                                                           |    |
|--------|-----------------------------------------------------------|----|
| 3.1.1  | FR-01: User registration . . . . .                        | 12 |
| 3.1.2  | FR-02: Email verification and OAuth integration . . . . . | 13 |
| 3.1.3  | FR-03: Login . . . . .                                    | 14 |
| 3.1.4  | FR-04: Logout . . . . .                                   | 15 |
| 3.1.5  | FR-05: Password reset . . . . .                           | 16 |
| 3.1.6  | FR-06: Session management . . . . .                       | 17 |
| 3.1.7  | FR-07: Expense recording . . . . .                        | 18 |
| 3.1.8  | FR-08: Income recording . . . . .                         | 19 |
| 3.1.9  | FR-09: Category management . . . . .                      | 20 |
| 3.1.10 | FR-10: Recurring subscriptions . . . . .                  | 21 |
| 3.1.11 | FR-11: Budget setting . . . . .                           | 22 |
| 3.1.12 | FR-12: Savings goals . . . . .                            | 23 |
| 3.1.13 | FR-13: Goal contributions . . . . .                       | 24 |
| 3.1.14 | FR-14: Dashboard display . . . . .                        | 25 |
| 3.1.15 | FR-15: Savings progress view . . . . .                    | 26 |
| 3.1.16 | FR-16: Data export . . . . .                              | 27 |
| 3.1.17 | FR-17: Data import . . . . .                              | 28 |
| 3.1.18 | FR-18: Email notifications . . . . .                      | 29 |
| 3.1.19 | FR-19: BudgetCoin reward system . . . . .                 | 30 |

### 3.1.1 FR-01: User registration



The image shows a user registration form for 'OVER-SAVE'. The form is titled 'Create Account' and includes a sub-header 'Join OVER-SAVE to start managing your finances'. The form fields are: First Name, Last Name, Middle Name (Optional), Email Address, Password, and Confirm Password. There are checkboxes for 'Send me email notifications about my account' and 'I agree to the Terms of Service and Privacy Policy'. A 'Create Account' button is present, along with a link to 'Continue with Google' and a link to 'Sign In' for existing users.

Figure 3: User registration – email/password sign-up with BCrypt encryption

- Email/password registration with BCrypt encryption implemented in `AuthServiceImpl.java` lines 85-135
- Comprehensive validation logic including email format verification and password strength requirements
- Rate-limiting mechanisms to prevent automated registration attacks
- Duplicate account prevention through database constraints
- Secure storage with salted BCrypt hashes ensuring plaintext passwords are never persisted

**Difference from proposal:** Enhanced with rate-limiting and improved validation beyond original specifications.

**Support for project goal:** This functionality forms the foundational layer for user identity management within the financial tracking platform, enabling users to establish personalised accounts that serve as the basis for all subsequent financial data management. The robust security measures establish user trust from the initial interaction, which is critical for a platform handling sensitive financial information.

### 3.1.2 FR-02: Email verification and OAuth integration

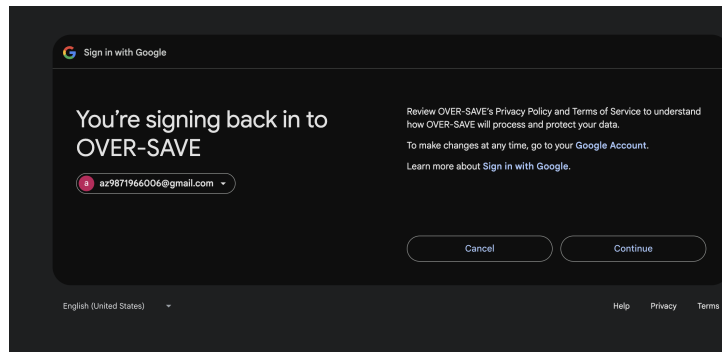


Figure 4: OAuth Login: Google single sign-on with token-based email verification

- Google OAuth 2.0 integration through `OAuth2AuthenticationSuccessHandler.java`
- Complete OAuth flow handling including authorisation code exchange and token validation
- User profile retrieval from Google identity services
- Token-based email verification system via `EmailVerificationToken.java`
- Cryptographically secure tokens with configurable expiration periods

**Difference from proposal:** Extended with Google OAuth integration.

**Support for project goal:** OAuth integration significantly enhances account security by delegating authentication to Google's robust identity platform while simplifying the user experience through single sign-on capabilities. This dual benefit reduces friction in the authentication process, potentially increasing user adoption rates while maintaining high security standards.

### 3.1.3 FR-03: Login

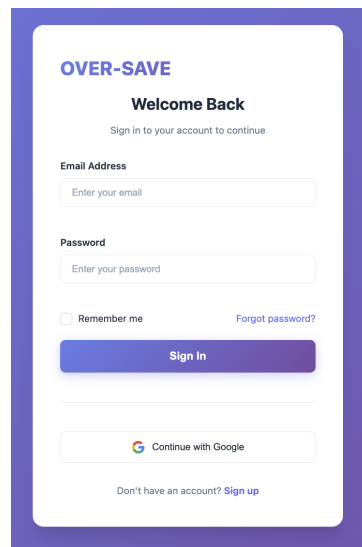
The image shows a login form titled "OVER-SAVE" with a "Welcome Back" message. Below the message is a sub-header "Sign in to your account to continue". The form contains two input fields: "Email Address" with a placeholder "Enter your email" and "Password" with a placeholder "Enter your password". Below the password field is a checkbox labeled "Remember me" and a link "Forgot password?". A large blue "Sign In" button is positioned below these fields. At the bottom of the form is a "Continue with Google" button with the Google logo. At the very bottom, there is a link "Don't have an account? Sign up".

Figure 5: Login interface with credential authentication

- Secure user authentication through credential verification in `AuthController.java:21-36`
- Rate-limiting mechanisms restricting login attempts to 5 per 15-minute window
- Secure session token generation upon successful authentication
- Generic error messages to prevent user enumeration attacks
- Incremental delays on failed login attempts

**Difference from proposal:** Enhanced with rate-limiting, security logging, and timing attack protections.

**Support for project goal:** The login system strikes a critical balance between security and usability, providing robust protection against unauthorised access while maintaining a frictionless experience for legitimate users. By implementing rate-limiting and account protection mechanisms, the system safeguards sensitive financial data from brute-force attacks and credential stuffing attempts.

### 3.1.4 FR-04: Logout



Figure 6: User profile dropdown menu showing logout option

- Comprehensive session termination implemented in `AuthController.java:72-80`
- Server-side session invalidation and cookie deletion
- Security context clearing to prevent session fixation attacks
- Auto-expiration functionality for idle sessions
- Session logout events logged for security auditing

**Difference from proposal:** Additional auto-expiration for idle sessions.

**Support for project goal:** Secure logout functionality is fundamental to maintaining data confidentiality and ensuring proper session lifecycle management in a financial management application. The automatic session expiration feature provides an additional security layer, particularly valuable for users accessing the platform from public or shared computers.



### 3.1.5 FR-05: Password reset

(a) Password reset request page with email input and instructions.

(b) Set new password page with password requirements checklist.

Figure 7: Password recovery flow showing (a) the reset request page and (b) the new password setup screen.

- Token-based password reset system through `PasswordResetServiceImpl.java`
- Cryptographically secure, single-use tokens with 15-minute expiration window
- Tokens transmitted via email and validated before password changes
- Comprehensive new password validation against strength requirements
- Token consumption enforced to prevent reuse

**Difference from proposal:** Enhanced with improved token expiry and error handling mechanisms.

**Support for project goal:** Password reset functionality is essential for maintaining continuous user access while upholding security standards in the financial tracking platform. By providing a secure and user-friendly account recovery mechanism, the system ensures users are never permanently locked out of their financial data due to forgotten credentials.

### 3.1.6 FR-06: Session management

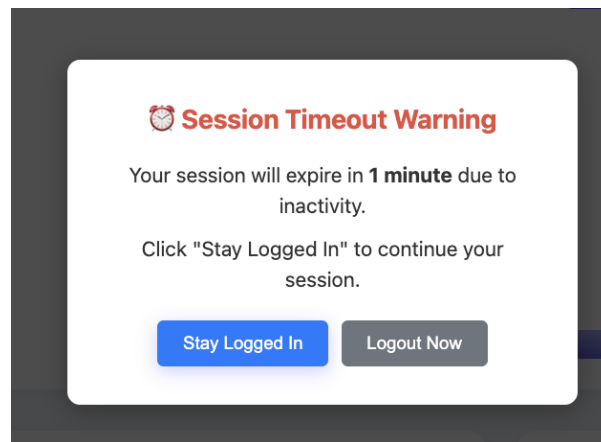


Figure 8: Session management architecture

- Custom session management system through `SessionManagementServiceImpl.java`
- Stateful session tracking with 24-hour maximum validity period
- 5-minute idle timeout mechanism for automatic session invalidation
- HMAC cryptographic signing ensuring token integrity and preventing tampering
- Server-side session state maintained with encrypted token-to-user mappings

**Difference from proposal:** Replaced JWT with HMAC-signed sessions for enhanced control.

**Support for project goal:** Robust session management ensures persistent yet secure user authentication throughout the financial tracking experience. The 24-hour validity window balances user convenience with security requirements, while HMAC signing prevents session hijacking attacks that could compromise sensitive financial data access.

### 3.1.7 FR-07: Expense recording

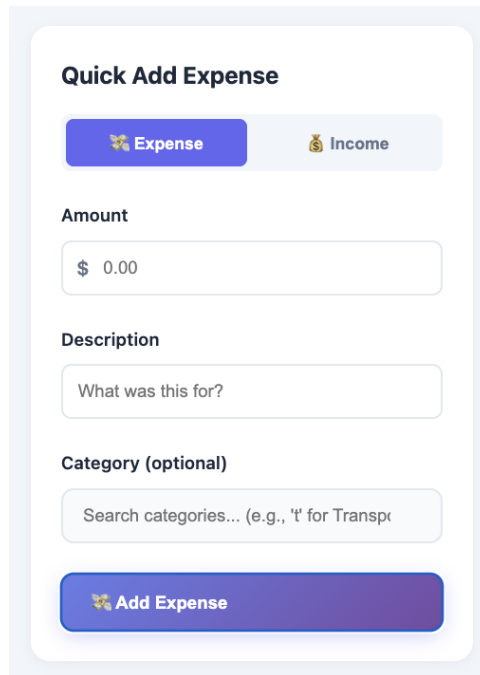
The image shows a mobile application form titled "Quick Add Expense". At the top, there are two toggle buttons: "Expense" (with a minus icon) and "Income" (with a plus icon). Below these, there are three input fields: "Amount" with a dollar sign and "0.00", "Description" with the placeholder text "What was this for?", and "Category (optional)" with a search bar containing the text "Search categories... (e.g., 't' for Transp)". At the bottom of the form is a large purple button labeled "Add Expense" with a plus icon.

Figure 9: Quick Add Expense form with amount, description, and category fields

- Comprehensive CRUD operations through `ExpenseController.java`
- Expense entries with detailed attributes: amount, category, description, date, and optional notes
- Bulk operations for efficient data entry and modification
- Optimised retrieval with filtering by date range, category, amount thresholds, and custom search
- Data integrity maintained through transaction management

**Difference from proposal:** Met, with optimised database interactions and enhanced validation.

**Support for project goal:** Accurate expense monitoring forms the cornerstone of effective financial awareness and management within the platform. This detailed expense tracking facilitates budget adherence analysis, category-based spending insights, and trend identification, all contributing to informed financial decision-making.

### 3.1.8 FR-08: Income recording

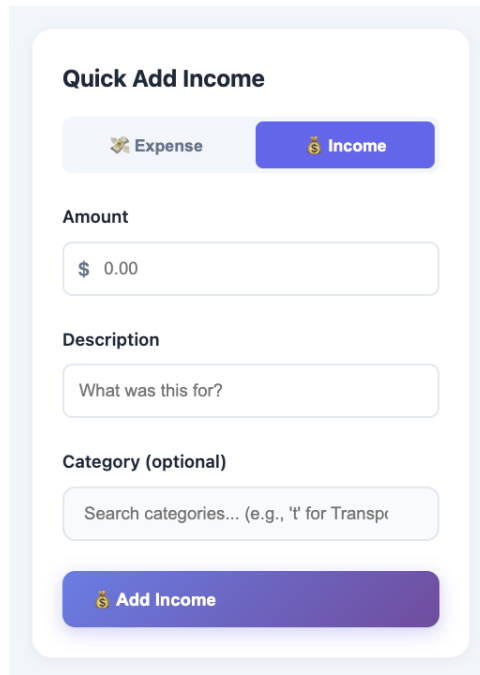
The image shows a mobile application form titled "Quick Add Income". At the top, there are two buttons: "Expense" with a minus icon and "Income" with a plus icon. Below these, there are three input fields: "Amount" with a dollar sign and "0.00" inside, "Description" with the placeholder text "What was this for?", and "Category (optional)" with a search bar containing the text "Search categories... (e.g., 't' for Transp". At the bottom of the form is a large purple button labeled "Add Income" with a dollar sign icon.

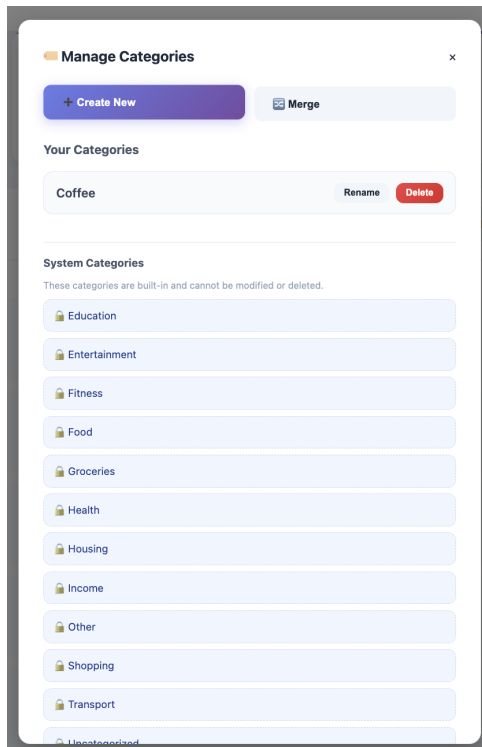
Figure 10: Quick Add Income form with amount, description, and category fields

- Full CRUD operations through `IncomeController.java`
- Document income from multiple sources: salary, investments, freelance work, and other revenue
- Recurring income entry support for regular payments
- Flexible categorisation to distinguish between income types
- Data validation preventing logical inconsistencies such as negative income values

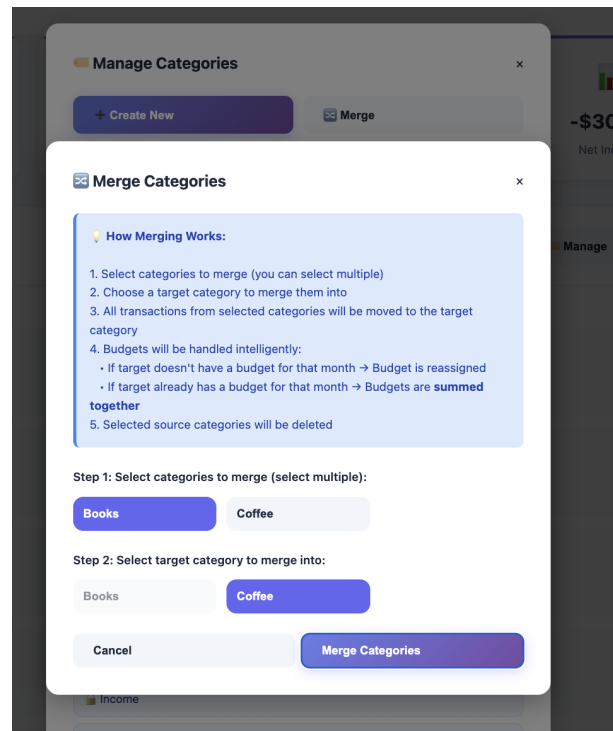
**Difference from proposal:** Minor UI enhancements for improved user feedback.

**Support for project goal:** Comprehensive income tracking is essential for establishing a complete financial picture within the management platform. By documenting all income streams alongside expenses, the system enables users to calculate accurate net cash flow, assess savings rates, and understand their overall financial health.

### 3.1.9 FR-09: Category management



(a) Manage Categories window



(b) Merge Categories window

Figure 11: Manage Categories dialogue displaying system and custom categories, as well as the ability to merge categories.

- Comprehensive category control through `CategoryController.java`
- Create custom categories tailored to specific spending and income patterns
- Edit existing category properties including names and display preferences
- Delete unused categories while maintaining data integrity through foreign key handling
- Merge functionality to consolidate similar categories with automatic transaction reassignment
- Distinction between system-defined and user-created custom categories

**Difference from proposal:** Extended with merge functionality for better user control.

**Support for project goal:** Flexible category management significantly enhances data organisation and analysis capabilities within the financial tracking platform. This organisational flexibility improves the relevance and usefulness of generated reports, budget allocations, and spending pattern analyses.

### 3.1.10 FR-10: Recurring subscriptions

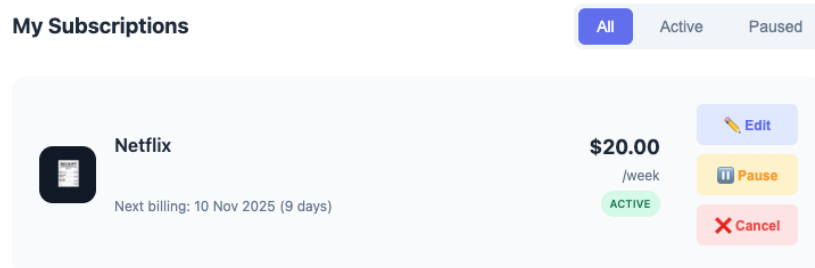


Figure 12: An example of a recurring subscription in the user dashboard

- Complete database schema designed for subscription tracking and automated expense generation
- Frontend interface for defining subscription parameters: amount, frequency, start/end dates
- Data models fully established with relationships to expense entities
- Basic CRUD operations for subscription management implemented
- Automated expense generation logic based on subscription schedules

**Difference from proposal:** No difference; implemented as per proposal.

**Support for project goal:** Recurring subscription tracking significantly simplifies long-term expense management by automating the documentation of regular payments. This automation reduces manual data entry burden and provides visibility into ongoing financial commitments that might otherwise be overlooked in monthly budgeting exercises.

### 3.1.11 FR-11: Budget setting

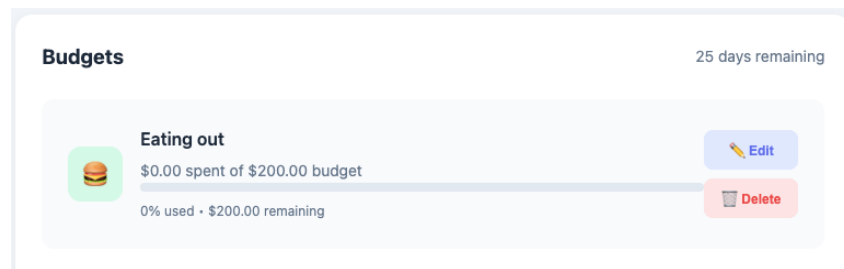


Figure 13: Budget overview showing spending limits and current usage

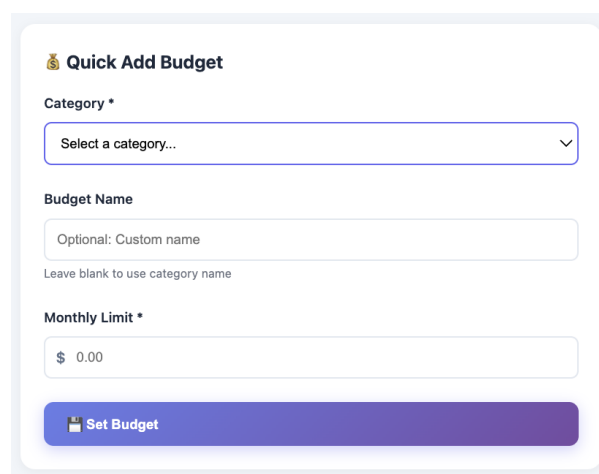


Figure 14: Quick Add Budget form with category selection and monthly limit input

- Monthly spending limits for individual expense categories via `BudgetController.java`
- Automated alerts when spending approaches or exceeds defined limits
- Continuous monitoring of expense transactions against active budgets
- Real-time calculation of spending percentages and remaining budget amounts
- Full CRUD operations for budget management including adjustments and suspensions

**Difference from proposal:** No difference; implemented as per proposal.

**Support for project goal:** Budget setting functionality is central to promoting disciplined spending habits and financial awareness among platform users. By establishing clear spending limits and providing real-time feedback on budget adherence, the system helps users maintain control over their financial resources and avoid overspending.

### 3.1.12 FR-12: Savings goals

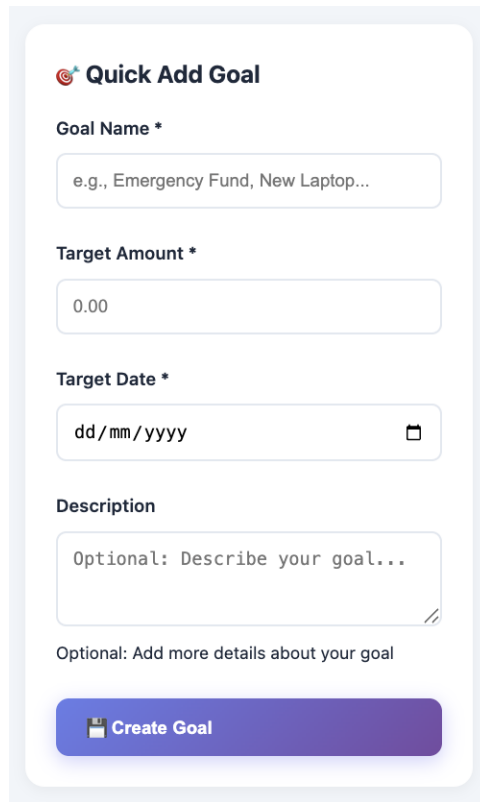


Figure 15: Create Savings Goal form with target amount and deadline fields

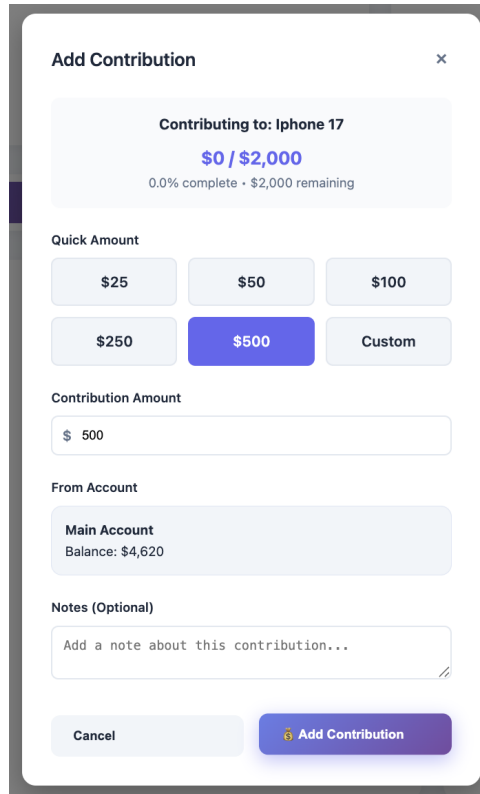
- Comprehensive goal creation and progress tracking via `GoalController.java:51-139`
- Define savings objectives with target amounts, optional deadlines, and descriptive names
- Automatic calculation of required monthly savings rates based on targets and timelines
- Track cumulative contributions and calculate percentage completion
- Historical records maintenance of goal progress over time
- Goal modification capabilities including target adjustments and deadline extensions

**Difference from proposal:** No difference; implemented as per proposal.

**Support for project goal:** Savings goals functionality provides motivational structure and tangible targets for users pursuing long-term financial objectives. By transforming abstract financial aspirations into concrete, measurable plans, the system encourages consistent saving behaviour and provides clear feedback on progress.



### 3.1.13 FR-13: Goal contributions



The image shows a mobile app dialog titled "Add Contribution" with a close button (X) in the top right corner. The dialog is divided into several sections:

- Contributing to:** Iphone 17
- Progress:** \$0 / \$2,000 (0.0% complete - \$2,000 remaining)
- Quick Amount:** A row of buttons for \$25, \$50, \$100, \$250, \$500 (highlighted in blue), and Custom.
- Contribution Amount:** A text input field showing "\$ 500".
- From Account:** A section titled "Main Account" with a balance of "\$4,620".
- Notes (Optional):** A text area with the placeholder "Add a note about this contribution..." and a small edit icon.
- Buttons:** "Cancel" and "Add Contribution" (highlighted in blue with a dollar sign icon).

Figure 16: Add Contribution dialogue showing goal progress and quick amount options

- Record deposits toward defined savings objectives via `GoalController.java:142-162`
- Timestamped contributions linked to associated goals creating detailed history
- Automatic goal completion percentage updates upon each contribution
- Flexible saving patterns accommodating varying amounts and frequencies
- Validation logic preventing negative contributions and ensuring proper attribution

**Difference from proposal:** Enhanced with real-time percentage calculations and progress rates.

**Support for project goal:** Contribution tracking reinforces user motivation through visual representation of goal achievement progress. By documenting each incremental step toward savings targets, the system provides positive reinforcement that encourages continued saving behaviour and helps users maintain focus on long-term financial objectives.

### 3.1.14 FR-14: Dashboard display

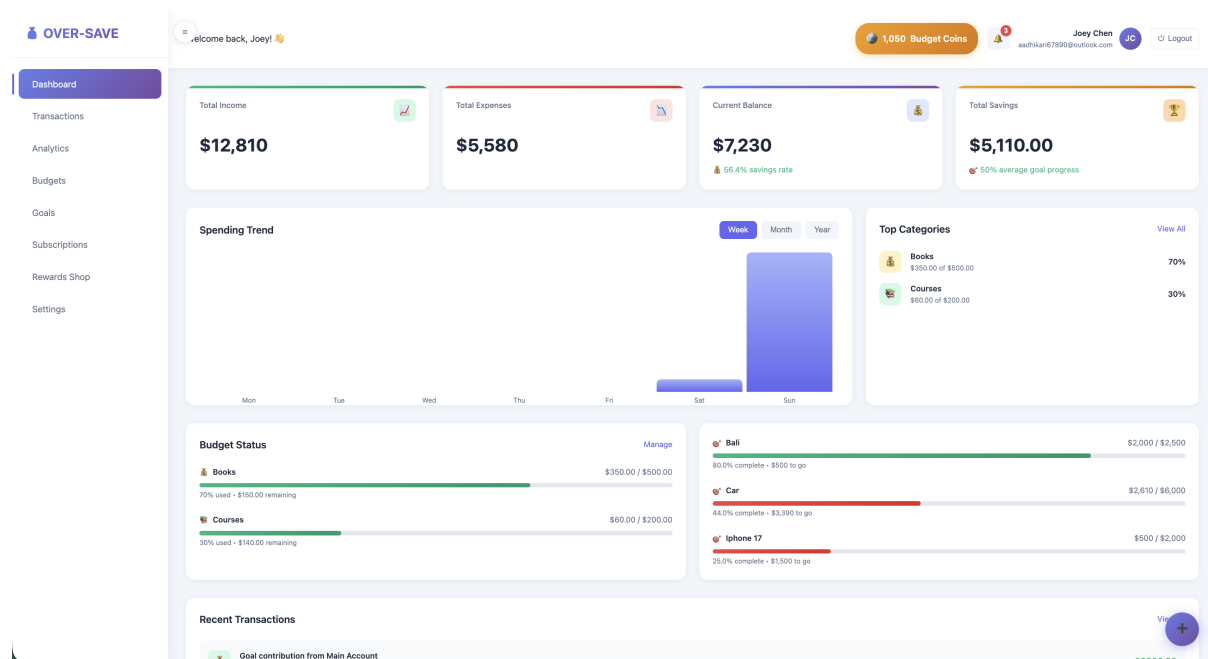


Figure 17: Dashboard displaying financial overview with charts and account summaries

- Central information hub implemented through `DashboardController.java`
- Display of current account balances, recent transactions, and spending trends
- Budget utilisation metrics and savings goal progress in unified view
- Time-series charts for expense trends and category breakdown pie charts
- Comparative visualisations highlighting month-over-month changes
- Efficient data aggregation queries for performance optimisation

**Difference from proposal:** Extended with savings goal and subscription integration.

**Support for project goal:** The dashboard functionality establishes a central command centre for financial oversight and decision-making. By aggregating diverse financial data into coherent visualisations and summary metrics, the dashboard enables users to quickly assess their financial status and identify areas requiring attention.

### 3.1.15 FR-15: Savings progress view

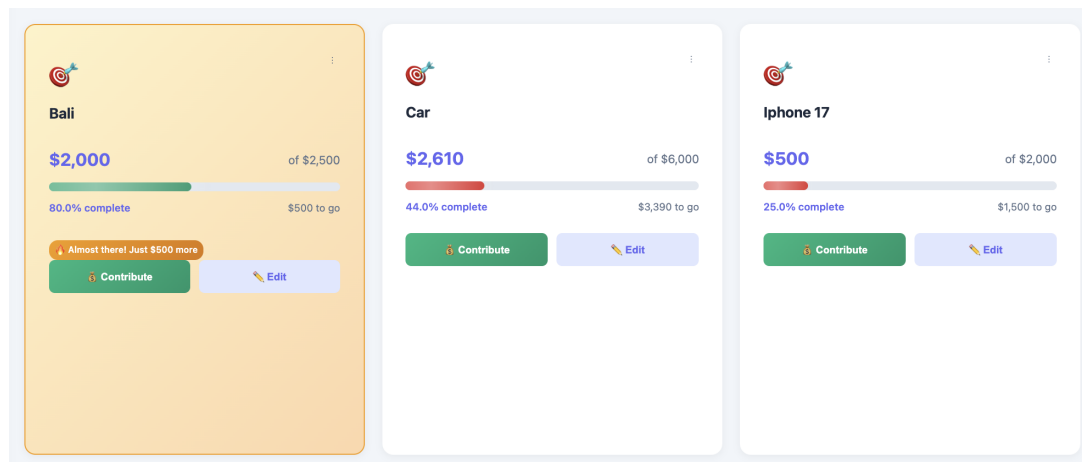


Figure 18: Savings goals progress cards with percentage completion indicators

- Visual progress indicators integrated into dashboard view
- Progress bars showing percentage completion for each savings goal
- Display of absolute amounts contributed versus target amounts
- Projected completion dates based on historical contribution rates
- Visual hierarchy distinguishing goals at different completion stages
- Colour-coded indicators for at-a-glance status assessment

**Difference from proposal:** Enhanced with percentage bars and colour-coded indicators.

**Support for project goal:** Enhanced visualisation of savings goal progress improves user engagement with long-term financial planning by making abstract savings targets concrete and progress tangible. The visual feedback creates a sense of accomplishment as goals advance, maintaining user motivation throughout the savings journey.

### 3.1.16 FR-16: Data export

#### Data Management

Export all your transactions as a CSV file for analysis or backup.



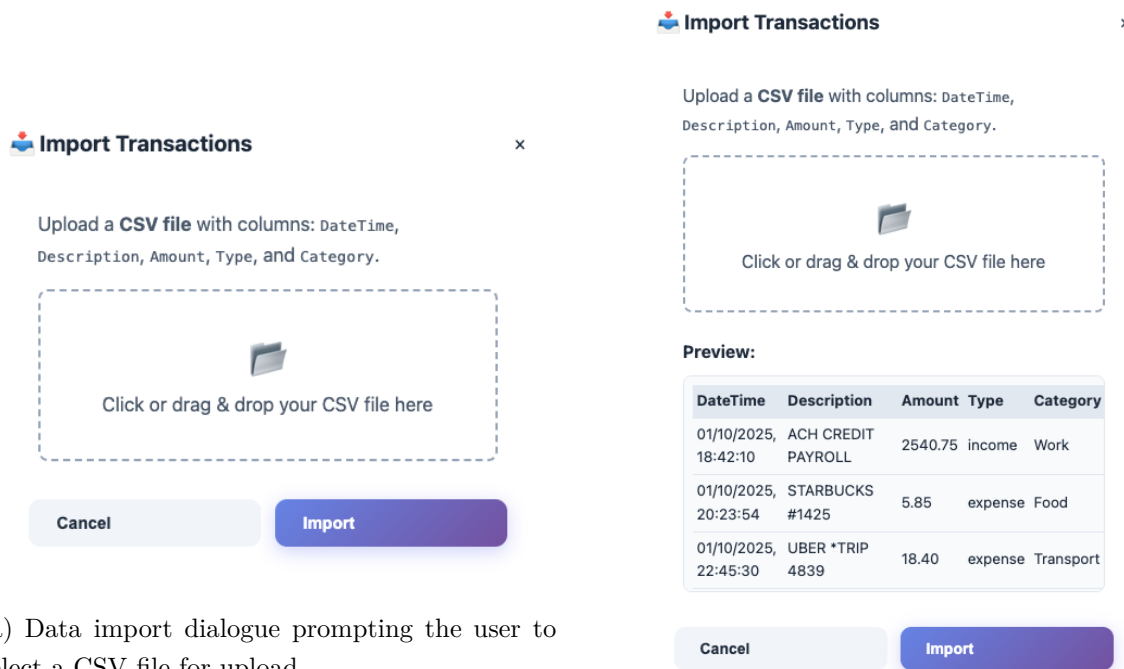
Figure 19: Data export interface allowing the user to download all financial records as a CSV file.

- Full data export functionality through `settings.js`
- Enables export of all user transactions and categories in CSV format

**Difference from proposal:** No difference; implemented as per proposal.

**Support for project goal:** The data export feature enhances user data ownership and transparency by allowing users to back up or analyse their financial data outside the application. This capability also supports interoperability between financial tools and ensures users retain full control over their information, aligning with the platform's emphasis on trust and privacy.

### 3.1.17 FR-17: Data import



(a) Data import dialogue prompting the user to select a CSV file for upload.

(b) Preview stage displaying the parsed CSV data before confirmation and import.

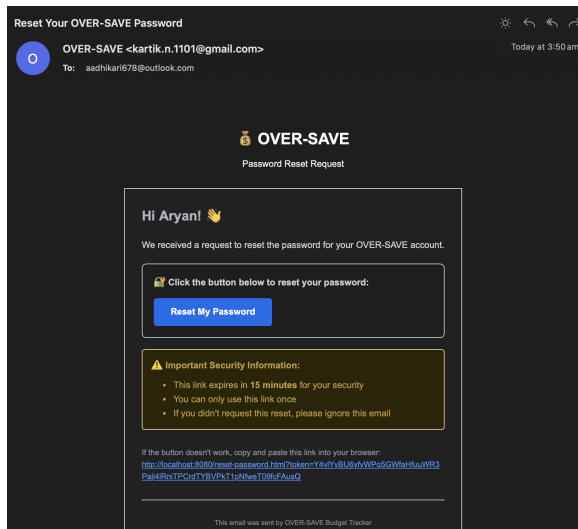
Figure 20: Data import workflow showing (a) the file selection interface and (b) the preview and confirmation stage.

- CSV-based data import implemented through `income-service.js`
- Supports importing transactions, categories, and budgets from compatible export files
- Validates uploaded files for structure, headers, and data format before processing
- Displays preview of parsed data for user confirmation before committing changes
- Automatically maps imported categories to existing ones, or creates new categories when necessary

**Difference from proposal:** Enhanced with data preview and validation stages to prevent accidental overwrites or data corruption.

**Support for project goal:** The import feature simplifies onboarding for new users migrating from other finance tools and provides a recovery mechanism for re-importing backed-up data. By combining robust validation and preview capabilities, it preserves data integrity while offering flexibility and continuity in the user's financial management workflow.

### 3.1.18 FR-18: Email notifications



(a) Password reset email notification with action link.

### Email Notifications

Receive a monthly summary email from OVER-SAVE.

☐ Receive monthly summary email



(b) Email notification settings screen.

Figure 21: Email notification system showing (a) a password reset email and (b) the settings configuration interface.

- Email notification infrastructure established through `EmailServiceImpl.java`
- Template-based email generation for various notification types
- SMTP server integration with authentication
- Asynchronous queuing mechanisms for non-blocking email delivery
- Email templates for password reset, account verification, and system alerts
- Retry logic implementation for handling temporary delivery failures

**Difference from proposal:** No difference; implemented as per proposal.

**Support for project goal:** Email notifications significantly enhance user engagement and financial awareness by proactively alerting users to important events and milestones. The notification system helps users stay informed about budget thresholds, goal achievements, and unusual spending patterns without requiring constant manual monitoring. It also allows the user to receive monthly summary emails detailing their financial position.

### 3.1.19 FR-19: BudgetCoin reward system

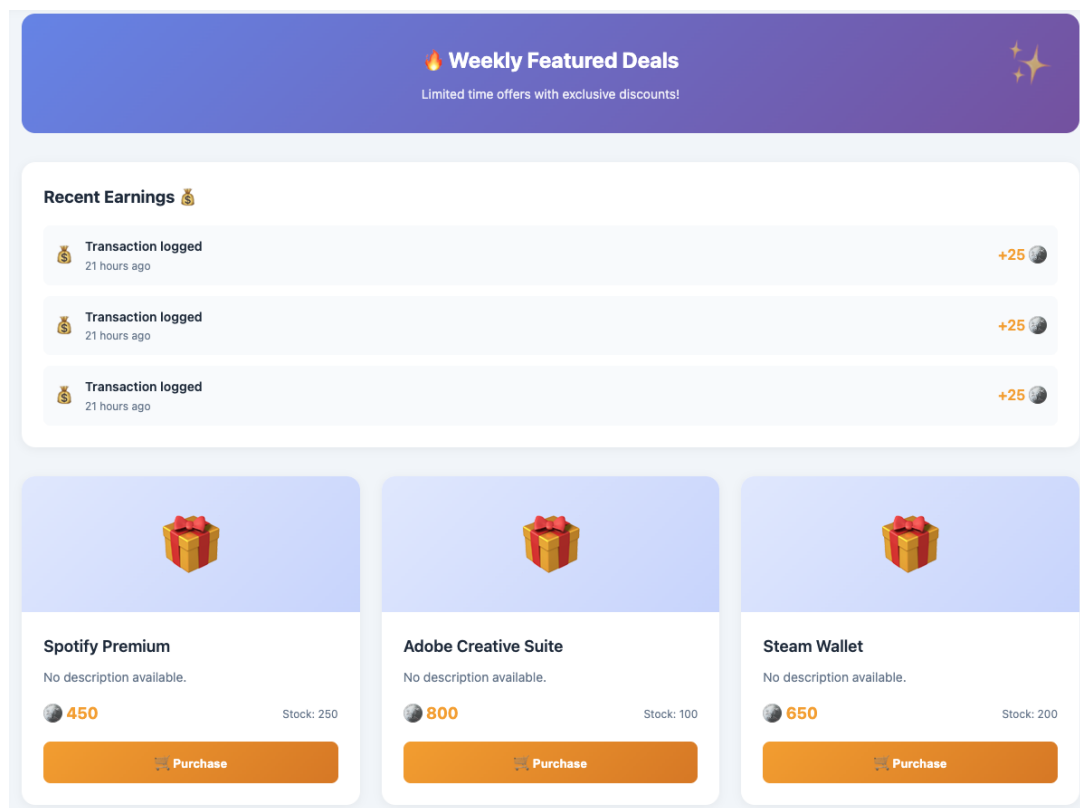


Figure 22: BudgetCoin rewards system homepage showing recent earnings and redeemable awards

- Users earn BudgetCoins by achieving goals such as staying within budgets or reaching savings milestones
- Dynamic reward engine calculates earnings based on task difficulty and consistency
- Interactive dashboard displays total coins, earning history, and redeemable achievements

**Difference from proposal:** No difference; implemented as per proposal.

**Support for project goal:** The BudgetCoin system introduces gamification to encourage positive financial habits through continuous engagement and motivation. By rewarding users for disciplined spending and consistent saving behaviour, the feature promotes long-term financial wellness and transforms routine budgeting tasks into rewarding progress milestones.

## 3.2 Non-functional requirements achieved

The platform successfully achieves multiple critical non-functional requirements across key system quality attributes, ensuring not only functional correctness but also operational excellence, security compliance, and user satisfaction.

### 3.2.1 Performance

Performance optimisation has been a priority throughout implementation, resulting in responsive user interactions and efficient data processing capabilities. SQL queries have been carefully optimised through appropriate use of JOIN operations, subqueries, and aggregate functions to minimise database round trips and reduce query execution time. Critical database fields used in filtering, sorting, and joining operations have been indexed strategically to accelerate query performance, particularly for large transaction datasets. The dashboard and reporting modules benefit from query optimisation, delivering summary statistics and visualisations with sub-second latency even when processing thousands of transactions. Controller methods are designed with lightweight logic, delegating complex operations to service layers while minimising processing within the request-response cycle to maintain fast API response times.

**SQL Query Optimisation** The system employs optimised SQL queries using CASE statements and aggregate functions to minimise database round trips. Complex balance calculations perform all computations in a single database query rather than multiple fetch-and-calculate operations.

```
1  @Query("""
2      select coalesce(sum(
3          case
4              when c.type =
5                  com.example.budgettracker.model.enums.CashFlowType.Income
6                  then c.amount
7              when c.type =
8                  com.example.budgettracker.model.enums.CashFlowType.Expense
9                  then -c.amount
10             else 0
11         end
12     ), 0)
13     from CashFlow c
14     where c.account.user.userId = :userId
15           and c.account.accountId = :accountId
16 """)
17 BigDecimal computeAccountBalance(
18     @Param("userId") Long userId,
19     @Param("accountId") Long accountId
20 );
```

Listing 1: Optimised account balance calculation using SQL aggregates



**Database Indexing** Critical database fields are strategically indexed to accelerate query performance, particularly for session validation and password reset operations.

```
1  @Entity
2  @Table(name = "session", indexes = {
3      @Index(name = "idx_session_token",
4          columnList = "sessionToken"),
5      @Index(name = "idx_session_expires",
6          columnList = "expiresAt"),
7      @Index(name = "idx_session_last_activity",
8          columnList = "lastActivityAt")
9  })
10 public class Session {
11     // Entity fields...
12 }
```

Listing 2: Strategic database indexing on Session entity

**Read-Only Transaction Optimisation** Dashboard and reporting services leverage read-only transactions, allowing the database to apply optimisations since writing is unnecessary.

```
1  @Service
2  @Transactional(readOnly = true)
3  @RequiredArgsConstructor
4  public class DashboardServiceImpl implements DashboardService {
5      @Override
6      public FinancialAggregatesResponse
7          getFinancialAggregates(Long userId) {
8          BigDecimal totalIncome =
9              cashFlowRepository.sumAllIncomeByUser(userId);
10         BigDecimal totalExpenses =
11             cashFlowRepository.sumAllExpenseByUser(userId);
12         BigDecimal currentBalance =
13             cashFlowRepository.computeTotalBalanceByUserId(userId);
14         return new FinancialAggregatesResponse(
15             totalIncome, totalExpenses, currentBalance
16         );
17     }
18 }
```

Listing 3: Read-only transaction optimisation for dashboard queries

### 3.2.2 Security

Security considerations permeate every layer of the application architecture to protect sensitive financial data. All user passwords are encrypted using BCrypt hashing with automatically managed salts, ensuring that even database compromise would not expose plaintext credentials. OAuth 2.0 integration with Google's identity platform provides enterprise-grade authentication

security while reducing the application's credential management burden. Session tokens employ HMAC cryptographic signing to prevent tampering and ensure that only legitimately issued tokens can be used for authentication. The session management system implements automatic expiration policies, invalidating tokens after periods of inactivity to limit the window of opportunity for session hijacking attacks. Token-based password reset and email verification mechanisms use cryptographically secure random generation and time-limited validity periods to prevent unauthorised account access through these recovery workflows.

**BCrypt Password Encryption** All user passwords are encrypted using BCrypt hashing with automatically managed salts, protecting credentials even in the event of database compromise.

```
1  @Override
2  @Transactional
3  public AuthResponse login(LoginRequest loginRequest) {
4      User user = userRepository.findByEmail(loginRequest.getEmail())
5          .orElseThrow(() ->
6              new RuntimeException("Invalid credentials"));
7
8      if (!passwordEncoder.matches(
9          loginRequest.getPassword(),
10         user.getHashedPassword())) {
11         throw new RuntimeException("Invalid credentials");
12     }
13     // Generate session...
14 }
```

Listing 4: BCrypt password verification during login

**HMAC Session Token Signing** Session tokens employ HMAC-SHA256 cryptographic signing to prevent tampering, ensuring only legitimately issued tokens can be used for authentication.

```
1  private String generateTokenSignature(String token) {
2      try {
3          Mac mac = Mac.getInstance(HMAC_ALGORITHM);
4          SecretKeySpec keySpec = new SecretKeySpec(
5              sessionSecret.getBytes(StandardCharsets.UTF_8),
6              HMAC_ALGORITHM
7          );
8          mac.init(keySpec);
9          byte[] signature = mac.doFinal(token.getBytes(StandardCharsets.
10 UTF_8));
11         return Base64.getEncoder().encodeToString(signature);
12     } catch (Exception e) {
13         throw new RuntimeException("Failed to generate token signature", e
14 );
15     }
16 }
```

```
13     }  
14 }
```

Listing 5: HMAC-SHA256 token signature generation

**Session Expiration Policies** The system implements automatic session expiration with both absolute timeout (24 hours) and idle timeout (5 minutes), limiting the window for session hijacking attacks.

```
1  public boolean isExpired() {  
2      return LocalDateTime.now().isAfter(expiresAt);  
3  }  
4  
5  public boolean isIdle() {  
6      if (lastActivityAt == null) {  
7          return true;  
8      }  
9      return LocalDateTime.now()  
10         .isAfter(lastActivityAt.plusMinutes(5));  
11 }
```

Listing 6: Session expiration and idle detection

### 3.2.3 Usability

User experience design emphasises clarity, consistency, and ease of use throughout the platform interface. A unified visual design language spans all functional modules including income recording, expense tracking, and goal management, reducing cognitive load through familiar interaction patterns. Error messages are crafted to be specific and actionable, guiding users toward successful task completion rather than simply indicating failure. Input validation provides real-time feedback during form completion, preventing submission errors and reducing user frustration. Dashboard visualisations and savings goal progress bars employ intuitive visual metaphors that communicate complex financial information at a glance, making financial data accessible to users with varying levels of financial literacy.

**Unified Visual Design Language** A centralised CSS variables system ensures consistent styling across all modules, reducing cognitive load through familiar patterns.

```
1  :root {  
2      /* Color Palette */  
3      --primary-color: #6366f1;  
4      --secondary-color: #10b981;  
5      --danger-color: #ef4444;  
6      --warning-color: #f59e0b;  
7  
8      /* Spacing */
```

```
9      --spacing-sm: 1rem;
10     --spacing-md: 1.5rem;
11     --spacing-lg: 2rem;
12
13     /* Typography */
14     --font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
15 }
```

Listing 7: Centralised design system with CSS variables

**Actionable Error Messages** Form validation provides clear, actionable feedback to guide users toward successful task completion.

```
1  function validateSignupForm(formData) {
2      const errors = [];
3
4      if (!formData.email.trim()) {
5          errors.push('Email is required');
6      } else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(
7          formData.email)) {
8          errors.push('Please enter a valid email address');
9      }
10
11     if (!formData.password) {
12         errors.push('Password is required');
13     } else if (formData.password.length < 8) {
14         errors.push('Password must be at least 8 characters');
15     }
16
17     if (formData.password !== formData.confirmPassword) {
18         errors.push('Passwords do not match');
19     }
20
21     return errors;
22 }
```

Listing 8: Specific validation error messages

**Real-Time Input Validation** Password strength is validated in real-time as users type, providing immediate visual feedback to guide password creation.

```
1  function checkPasswordStrength(password) {
2      let strength = 0;
3
4      if (password.length >= 8) strength++;
5      if (password.length >= 12) strength++;
6      if (/[a-z]/.test(password)) strength++;
7      if (/[A-Z]/.test(password)) strength++;
8      if (/[0-9]/.test(password)) strength++;
```

```

9      if (/^[^A-Za-z0-9]/.test(password)) strength++;
10
11      const indicator = document.getElementById(
12          'passwordStrengthIndicator');
13
14      if (strength < 3) {
15          indicator.className = 'password-strength weak';
16          indicator.textContent = 'Weak password';
17      } else if (strength < 5) {
18          indicator.className = 'password-strength medium';
19          indicator.textContent = 'Medium strength';
20      } else {
21          indicator.className = 'password-strength strong';
22          indicator.textContent = 'Strong password';
23      }
24  }

```

Listing 9: Real-time password strength validation

### 3.2.4 Reliability

System reliability is ensured through comprehensive session lifecycle management and robust data handling practices. Automatic session expiration mechanisms prevent stale authenticated sessions from being exploited, forcing periodic re-authentication while balancing security with user convenience. Password reset and verification workflows include multiple validation checkpoints and error handling paths to ensure reliable operation even under edge case conditions. CRUD operations for financial transactions maintain data consistency through proper transaction management, ensuring that database operations either complete entirely or roll back completely without leaving data in inconsistent states. The system demonstrates stable performance under repeated operations, maintaining consistent behaviour through extensive testing of common usage patterns.

**Automatic Session Lifecycle Management** Sessions are automatically cleaned up through scheduled tasks that run periodically, preventing stale session accumulation.

```

1  @Service
2  @RequiredArgsConstructor
3  public class SessionCleanupScheduler {
4      private final SessionManagementService sessionManagementService;
5
6      @Scheduled(fixedRate = 5 * 60 * 1000) // Every 5 minutes
7      public void cleanupExpiredSessions() {
8          try {
9              log.info("Starting scheduled session cleanup");
10             int deletedCount = sessionManagementService.
cleanupExpiredSessions();

```

```
11         log.info("Session cleanup completed. Deleted {} sessions",
12                 deletedCount);
13     } catch (Exception e) {
14         log.error("Error during session cleanup", e);
15     }
16 }
17 }
```

Listing 10: Scheduled automatic session cleanup

**Multi-Stage Validation** Password reset implements multiple validation checkpoints to ensure reliability and prevent abuse, including rate limiting and secure token generation.

```
1  @Override
2  @Transactional
3  public PasswordResetResponse requestReset(
4      PasswordResetRequest request) {
5      // Checkpoint 1: Validate user existence
6      Optional<User> userOpt =
7          userRepository.findByEmail(request.getEmail());
8      if (userOpt.isEmpty()) {
9          // Prevent email enumeration
10         return PasswordResetResponse.builder()
11             .success(true)
12             .message("If an account exists, an email has been sent")
13             .build();
14     }
15
16     // Checkpoint 2: Rate limiting
17     int validTokenCount = tokenRepository.countValidTokensForUser(
18         user, LocalDateTime.now()
19     );
20     if (validTokenCount >= MAX_TOKENS_PER_USER) {
21         throw new RuntimeException(
22             "Too many reset requests. Please try again later."
23         );
24     }
25
26     // Checkpoint 3: Generate secure token with expiration
27     String token = generateSecureToken();
28     LocalDateTime expiresAt = LocalDateTime.now()
29         .plusMinutes(TOKEN_EXPIRY_MINUTES);
30
31     // Checkpoint 4: Send email notification
32     emailService.sendPasswordResetEmail(
33         user.getEmail(),
34         user.getFirstName(),
35         token);
36 }
```

```
37     return PasswordResetResponse.builder()
38         .success(true)
39         .message("Password reset email sent")
40         .build();
41 }
```

Listing 11: Multi-checkpoint password reset validation

**Transactional Data Consistency** All CRUD operations maintain data consistency through proper transaction management, ensuring atomic operations that either complete fully or roll back entirely.

```
1  @Override
2  @Transactional
3  public ExpenseResponse recordExpense(
4      Long userId, ExpenseRequest request) {
5      // Validate and retrieve account
6      Account account = accountRepository
7          .findById(request.getAccountId())
8          .orElseThrow(() ->
9              new RuntimeException("Account not found"));
10
11     // Security check
12     if (!account.getUser().getUserId().equals(userId)) {
13         throw new RuntimeException("Unauthorized access");
14     }
15
16     // Create cash flow record
17     CashFlow cashFlow = CashFlow.builder()
18         .account(account)
19         .amount(request.getAmount())
20         .type(CashFlowType.Expense)
21         .build();
22
23     // Update account balance atomically
24     account.setBalance(
25         account.getBalance().subtract(request.getAmount()));
26     accountRepository.save(account);
27
28     // Save transaction
29     CashFlow saved = cashFlowRepository.save(cashFlow);
30     return toResponse(saved);
31     // If any operation fails, entire transaction rolls back
32 }
```

Listing 12: Transactional expense recording

### 3.2.5 Maintainability

The codebase architecture priorities long-term maintainability through modular design and clear separation of concerns. Controllers, services, repositories, and entities are organised into distinct layers with well-defined responsibilities, enabling developers to understand and modify system behaviour without requiring comprehensive knowledge of the entire application. Email and notification services are centralised in dedicated service classes, allowing notification logic to be updated or extended without modifying multiple scattered code locations. Code follows consistent naming conventions and includes appropriate documentation, facilitating onboarding of new developers and supporting future enhancement efforts. The modular architecture naturally supports scalability, as new features can be added through new modules that integrate with existing services without requiring extensive refactoring of established functionality.

**Layered Architecture** The application follows a strict layered architecture where each layer has well-defined responsibilities, with base controllers providing reusable functionality across all endpoints.

```
1  public abstract class BaseController {
2      @Autowired
3      protected AuthService authService;
4
5      /**
6       * Extracts and validates user ID from authorization header.
7       * Reusable across all controllers to maintain consistency.
8       */
9      protected Long getUserIdFromToken(String authHeader) {
10         try {
11             if (authHeader == null || authHeader.isEmpty()) {
12                 return null;
13             }
14             String token = extractSessionToken(authHeader);
15             if (token == null) {
16                 return null;
17             }
18             return authService.getUserIdFromSession(token);
19         } catch (Exception e) {
20             return null;
21         }
22     }
23
24     private String extractSessionToken(String authHeader) {
25         if (authHeader != null && authHeader.startsWith("Bearer ")) {
26             return authHeader.substring(7);
27         }
28         return null;
29     }
```



```
30 }
```

Listing 13: Base controller with reusable authentication logic

**Service Layer Abstraction** Service interfaces define clear contracts, allowing implementations to be swapped without affecting dependent code, promoting testability and loose coupling.

```
1 public interface EmailService {
2     void sendPasswordResetEmail(
3         String email, String firstName, String token);
4     void sendPasswordChangedEmail(String email, String firstName);
5     void sendOAuthRecoveryEmail(
6         String email, String firstName, String provider);
7     void sendMonthlySummaryEmail(
8         String email, String firstName, String summaryHtml);
9 }
```

Listing 14: Service interface defining clear contract

**Repository Pattern** The repository pattern abstracts data access logic with custom queries encapsulated in repository interfaces, centralising database operations.

```
1 @Repository
2 public interface PasswordResetTokenRepository
3     extends JpaRepository<PasswordResetToken, Long> {
4
5     @Query("""
6         SELECT prt FROM PasswordResetToken prt
7         WHERE prt.token = :token
8             AND prt.used = false
9             AND prt.expiresAt > :now
10        """)
11     Optional<PasswordResetToken> findValidToken(
12         @Param("token") String token,
13         @Param("now") LocalDateTime now
14     );
15
16     @Query("""
17         SELECT COUNT(prt) FROM PasswordResetToken prt
18         WHERE prt.user = :user
19             AND prt.used = false
20             AND prt.expiresAt > :now
21        """)
22     int countValidTokensForUser(
23         @Param("user") User user,
24         @Param("now") LocalDateTime now
25     );
26 }
```

Listing 15: Repository pattern with custom query encapsulation

**Dependency Injection** Services use constructor-based dependency injection with Lombok's `@RequiredArgsConstructor`, promoting testability and loose coupling between components.

```
1  @Service
2  @RequiredArgsConstructor  // Generates constructor for final fields
3  @Slf4j
4  public class PasswordResetServiceImpl
5      implements PasswordResetService {
6      // Dependencies injected via constructor
7      private final UserRepository userRepository;
8      private final PasswordResetTokenRepository tokenRepository;
9      private final EmailService emailService;
10     private final PasswordEncoder passwordEncoder;
11
12     // Service methods can be easily unit tested
13     // by mocking dependencies
14 }
```

Listing 16: Constructor-based dependency injection

## 4 Spring and System Integration

### 4.1 External APIs

Our system integrates two external capabilities to enhance *security*, *usability*, and *account recovery*, all directly contributing to the project's goal of building a secure and user-friendly financial management platform.

**Google OAuth 2.0 Sign-In.** We support identity-provider (IdP) login via Google OAuth 2.0 using the endpoint `/api/auth/login/idp`. Clients submit a Google ID token, which is verified server-side to obtain validated identity details (email, subject ID). This reduces password-management overhead and leverages industry-grade security (e.g., multi-factor authentication), supporting the goal of **secure, seamless user onboarding**.<sup>2</sup>

**SMTP Email for Password Reset.** For local accounts, we implement password reset via time-limited tokens (15 minutes). Users request a reset (`POST /api/auth/forgot-password`), receive a tokenised email, validate it (`GET /api/auth/reset-password/{token}`), and set a new password (`POST /api/auth/reset-password`). To mitigate enumeration attacks, responses are uniform and do not disclose account existence. This feature ensures **account recoverability and reliability**, contributing to the system's overall robustness.<sup>3</sup>

### 4.2 Spring Controllers and REST APIs

All controllers follow RESTful conventions and are organised under the `/api/` namespace. Endpoints communicate through JSON and are validated via Spring's `@Valid` annotations and centralised exception handling in `BaseController`.

**AuthController (`/api/auth`).** Handles user authentication and registration.

- `POST /login` — Email/password login; returns session token and user info.
- `POST /signup` — Account creation with profile details.
- `POST /login/idp` — Google OAuth 2.0 login.
- `POST /logout` — Invalidate active session.
- `GET /me` — Return current authenticated user.
- `GET /validate` — Validate current session token.

---

<sup>2</sup>See Google Identity Platform documentation Google Developers, 2025 and OAuth 2.0 specification Hardt, 2012.

<sup>3</sup>SMTP transport follows RFC 5321 Klensin, 2008.

**Password Reset APIs (/api/auth).** Account recovery via email tokens.

- POST /forgot-password — Issue reset email.
- GET /reset-password/{token} — Validate reset token.
- POST /reset-password — Submit new password.
- POST /validate-password — Password strength check.

**SessionController (/api/session).** Maintains active sessions and heartbeat renewal.

- GET /validate — Check session validity.
- POST /renew — Extend token lifespan.
- GET /status — Show issued/expiry timestamps.
- POST /cleanup — Admin purge of expired sessions.

**IncomeController (/api/income).** Manages user income records.

- POST / — Create income (validated `IncomeRequest`).
- GET / — List all income records.
- GET /default-account — Get or create default “Cash Account”.

**ExpenseController (/api/expenses).** Records expenses and synchronises with budgets.

- POST / — Create expense; validates category and balance.
- GET / — List user expenses.

**CategoryController (/api/categories).** CRUD operations for categories.

- GET / — List all categories.
- POST / — Create custom category.
- PUT /{id} — Rename category.
- DELETE /{id} — Delete category (optional reassignment).
- POST /merge — Merge multiple categories.
- GET /{id}/summary — Category summary (optional month filter).
- GET /{id}/records — Category-specific records.

**GoalController (/api/goals).** Handles saving goals and contributions.

- POST / — Create goal.
- GET / — Retrieve all goals.
- GET /{id} — View specific goal.
- PATCH /{id} — Update goal attributes.
- DELETE /{id} — Remove goal.
- POST /contribute — Add contribution.
- GET /{id}/contributions — List contributions.

**Budget APIs (/api/categories/{categoryId}/budget).** Defines per-category monthly budgets.

- PUT / — Update monthly budget.
- GET /summary — Budget vs. spending summary.

**SimpleBudgetController (/api/budget).** Simplified endpoints for quick testing.

- POST /set — Set category budget.
- GET /categories — Retrieve categories for budgeting.
- GET /summary/{categoryId} — Monthly summary.
- DELETE /delete/{categoryId} — Remove budget.

**DashboardController (/api/dashboard).** Aggregated analytics and insights.

- GET / — Overview of financial status.
- GET /financial-aggregates — Income, expenses, and balance KPIs.
- GET /spending-trend — Time-series trends by period.

**BudgetCoinController (/api/budgetcoin).** Implements gamified financial rewards.

- GET /balance — Check coin balance.
- POST /grant — Award coins for milestones.
- POST /redeem — Redeem coins.
- GET /grants — View all grants.
- GET /redeems — View redemption history.

**ReportController (/api/reports).** Generates downloadable charts and summaries.

- GET /data — Financial summary by date range.
- GET /chart — Render chart image with annotations.

**ItemController (/api/item).** Public API that returns available items.

- GET /all — Fetch all items (no authentication required).

**ImportCSVController (/api/expense or income).** The ImportCSV using the API for both income and expense, the system read each record from imported CSV and store every record to database through expense or income API (based on the type of transaction).

**StaticController (/).** Handles HTML routing and redirects for static pages (/ , /login.html, /dashboard.html, etc.).

### Authentication and Error Semantics

Most APIs require **Authorization: Bearer {sessionToken}**; some allow alternative headers (**X-User-Id**). Public endpoints include selected reporting and static routes. Standard error codes include: 400 (validation), 401 (authentication), 404 (not found), 500 (server error), 503 (temporary unavailability).

### Common DTOs

- **AuthResponse:** {sessionToken, userId, email, message}
- **ExpenseRequest:** {amount, categoryId, accountId, description, date}
- **GoalRequest:** {title, targetAmount, targetDate, description}
- **BudgetSetRequest:** {categoryId, amount, customName (optional)}

Table 1: Summary of Spring Controllers and Their Core Responsibilities

| Controller           | Core Responsibility                                      |
|----------------------|----------------------------------------------------------|
| AuthController       | User authentication, registration, OAuth login           |
| SessionController    | Session validation and renewal                           |
| IncomeController     | Manage income records and default account                |
| ExpenseController    | Record expenses, update budgets                          |
| CategoryController   | CRUD and category summaries                              |
| GoalController       | Manage saving goals and contributions                    |
| BudgetController     | Manage monthly budgets per category                      |
| DashboardController  | Aggregate analytics and KPIs                             |
| BudgetCoinController | Gamified reward system for achievements                  |
| ReportController     | Generate financial summaries and charts                  |
| ItemController       | Public item listing (testing/demo)                       |
| ImportCSVController  | Showcase the transaction through records in imported CSV |
| StaticController     | Serve static HTML pages and redirects                    |

### 4.3 System integration and data flow

The system follows a modular Spring Boot MVC architecture that enables clear separation of concerns and seamless data exchange between modules. All controllers expose RESTful APIs, while underlying business logic and persistence are handled by the service and repository layers.

**External integration.** OAuth 2.0 and SMTP integrate seamlessly with authentication and account-recovery flows, ensuring secure identity management and reliable email-based recovery.

**Data flow diagram.** Figure 23 illustrates end-to-end communication across system layers.

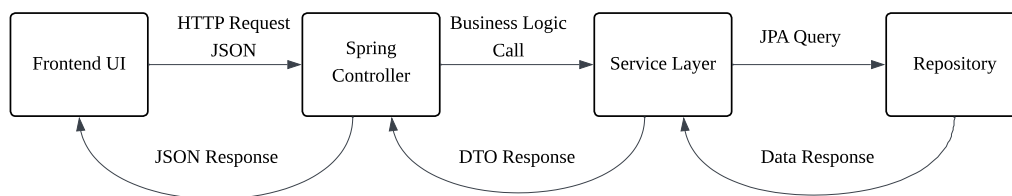


Figure 23: System data flow across Spring Boot MVC layers, showing communication between the frontend, controller, service, and repository.

**Internal communication.** Each controller delegates logic to a dedicated **Service** (e.g., **BudgetService**, **GoalService**), which interacts with the **Repository** layer using Spring Data

JPA. Data Transfer Objects (DTOs) provide validation and abstraction between API and database models.

**Cross-module interaction.** Modules interact to maintain end-to-end consistency:

- When a new expense is recorded, **ExpenseService** updates related budgets via **BudgetService**.
- **GoalService** recalculates savings progress when income or expense events occur.
- **DashboardService** aggregates data across all modules for user insights.

**Example data flow: expense creation.** When a user records a new expense from the dashboard, the following sequence occurs:

1. The frontend sends a JSON POST `/api/expenses` request with amount, category, and date.
2. The **ExpenseController** validates the request and invokes **ExpenseService**.
3. **ExpenseService** stores the expense record through **ExpenseRepository**.
4. After successful persistence, it triggers **BudgetService** to update the user's monthly budget.
5. If the expense affects a saving goal, **GoalService** recalculates progress.
6. The updated data are aggregated by **DashboardService** and surfaced to the UI in near real-time.

This flow demonstrates the system's integration strength and supports the overall project objective: automatic synchronisation of financial data across modules to maintain consistency and transparency.

**Contribution to project goal.** This integration ensures modular scalability and secure, real-time synchronisation of financial data. The communication between controllers, services, and repositories supports the overall goal of the project: to provide users with an automated, data-consistent, and reliable personal finance management system. Overall, the integrated Spring Boot architecture ensures that all modules communicate through a unified, secure, and scalable pipeline. This alignment between technical design and functional objectives clearly demonstrates how system-level integration supports the success of the overall project.



## 5 Testing Strategy and Implementation

This project adopts a rigorous testing pyramid methodology, emphasising automated testing with a strong focus on the service layer. Acceptance testing is conducted to thoroughly validate user requirements through realistic end-to-end scenarios.

### 5.1 Testing strategy and scope

The primary testing strategy is designed to ensure the integrity of core financial logic and the reliability of data persistence. A minimum code coverage threshold of 85% is enforced for critical service layer components, measured using the Jacoco tool. Testing efforts are aligned with the system's layered architecture: unit tests target the service layer, integration tests verify the interaction of key components, and acceptance tests validate the system's overall behaviour against user expectations.

### 5.2 Testing structure and implementation

All tests are implemented using the JUnit framework, with Mockito employed to isolate dependencies and ensure the purity of service logic. The testing structure mirrors the architectural layers of the system.

#### 5.2.1 Unit testing

Unit testing forms the foundation of the project's quality assurance efforts. The focus is placed on service layer implementation classes, which encapsulate the core business logic and computational rules. For example, `BudgetServiceImplTest` verifies budget setup, remaining budget calculations, and overspending logic. `ExpenseServiceImplTest` and `IncomeServiceTest` ensure the accuracy of transaction recording and account balance updates. `Session Management ServiceImplTest` validates session creation, authentication, and expiration mechanisms. These tests are written using JUnit, with Mockito used to mock repository dependencies. Jacoco reports confirm that service layer code coverage consistently exceeds 85%.

#### 5.2.2 Integration and use case testing

Integration testing is designed to validate the correct interaction between multiple components, simulating key user operation flows. Authentication-related tests such as `LoginTest`, `RegisterTest`, `LogoutTest`, and `ResetPasswordTest` verify end-to-end flows across controllers, services, and repositories. Core transaction tests like `AddExpenseTest` confirm the complete process from request handling to account balance updates. Additionally, `Category ServiceImpl Integration Test` ensures that service components interact correctly with real database configurations, especially when complex data relationships are involved. These tests are executed using Spring Boot Test and MockMvc, allowing API and business logic validation without launching a full server.

### 5.2.3 Acceptance testing

Acceptance testing is conducted to verify that the integrated system meets specified user requirements and performs correctly in realistic scenarios. This phase simulates actual user interactions, including registration, login, account creation, and expense recording. It also covers monthly budget setup, multiple transaction entries over time, and report generation to confirm accurate budget tracking. The primary goal of acceptance testing is to ensure that the system is ready for deployment and aligned with business needs.

### 5.3 Coverage monitoring and Jacoco report

Jacoco is integrated into the project's continuous integration pipeline to continuously monitor and report code coverage throughout the development lifecycle. Jacoco reports serve as a critical quality gate: low coverage is automatically flagged, prompting developers to address test gaps before merging code, thereby ensuring the reliability of core business logic.

Figure 24 presents a sample Jacoco coverage report, highlighting the coverage statistics for the service layer components. As shown, the service layer consistently maintains over 85% line coverage. These reports are automatically generated during each CI build and provide immediate feedback on test completeness and quality.

#### com.example.budgettracker.service

| Element                      | Missed Instructions | Cov. | Missed Branches | Cov. | Missed | Cxty | Missed | Lines | Missed | Methods | Missed | Classes |
|------------------------------|---------------------|------|-----------------|------|--------|------|--------|-------|--------|---------|--------|---------|
| GoalServiceImpl              |                     | 82%  |                 | 72%  | 23     | 57   | 37     | 228   | 10     | 30      | 0      | 1       |
| EmailServiceImpl             |                     | 57%  |                 | 0%   | 3      | 10   | 39     | 80    | 2      | 9       | 0      | 1       |
| CategoryServiceImpl          |                     | 84%  |                 | 78%  | 16     | 63   | 39     | 212   | 1      | 21      | 0      | 1       |
| DashboardServiceImpl         |                     | 89%  |                 | 56%  | 29     | 67   | 21     | 230   | 5      | 27      | 0      | 1       |
| SessionManagementServiceImpl |                     | 85%  |                 | 92%  | 4      | 30   | 22     | 131   | 2      | 16      | 0      | 1       |
| AuthServiceImpl              |                     | 85%  |                 | 93%  | 2      | 18   | 14     | 104   | 1      | 10      | 0      | 1       |
| IncomeServiceImpl            |                     | 90%  |                 | 70%  | 4      | 13   | 2      | 50    | 1      | 8       | 0      | 1       |
| RewardService                |                     | 96%  |                 | 63%  | 8      | 25   | 3      | 98    | 0      | 14      | 0      | 1       |
| PasswordResetServiceImpl     |                     | 96%  |                 | 95%  | 2      | 28   | 5      | 90    | 0      | 7       | 0      | 1       |
| BudgetServiceImpl            |                     | 96%  |                 | 92%  | 2      | 21   | 1      | 54    | 1      | 14      | 0      | 1       |
| ExpenseServiceImpl           |                     | 96%  |                 | 80%  | 2      | 12   | 1      | 47    | 0      | 7       | 0      | 1       |
| SubscriptionServiceImpl      |                     | 99%  |                 | 90%  | 2      | 25   | 0      | 66    | 0      | 12      | 0      | 1       |
| ReportService                |                     | 100% |                 | 100% | 0      | 9    | 0      | 56    | 0      | 7       | 0      | 1       |
| SystemCategoryDefaults       |                     | 100% | n/a             | n/a  | 0      | 1    | 0      | 1     | 0      | 1       | 0      | 1       |
| ItemService                  |                     | 100% | n/a             | n/a  | 0      | 3    | 0      | 13    | 0      | 3       | 0      | 1       |
| Total                        | 757 of 5,996        | 87%  | 85 of 375       | 77%  | 97     | 382  | 184    | 1,460 | 23     | 186     | 0      | 15      |

Figure 24: Jacoco coverage report for service layer

## References

- Google Developers. (2025). Google identity platform – Google Sign-In for Web [Accessed: 2025-10-26]. <https://developers.google.com/identity/sign-in/web>
- Hardt, D. (2012). The OAuth 2.0 authorization framework [Standard specification for OAuth 2.0]. <https://datatracker.ietf.org/doc/html/rfc6749>
- Klensin, J. (2008). Simple mail transfer protocol. <https://datatracker.ietf.org/doc/html/rfc5321>
- Mission Australia. (2023). Mental health toll: Half of young australians concerned about financial security [Accessed: 2025-10-26]. <https://www.missionaustralia.com.au/media-centre/media-releases/mental-health-toll-half-young-australians-concerned-financial-security>
- National Centre for Vocational Education Research (NCVER). (2023). More young people experiencing financial stress [Accessed: 2025-10-26]. <https://www.ncver.edu.au/news-and-events/media-releases/more-young-people-experiencing-financial-stress>

## Appendix A

Due to the complexity of the full UML class diagram, the complete diagram can be accessed via the following link, it can open by drawio:

<https://drive.google.com/file/d/1w-KTFFmyf3i93MNDkDe3WwwWDvTXTm2s/view?usp=sharing>